Regression — Commentary

Lesson 3 — Technologies for Artificial Intelligence

Fabio Antonini — Università degli Studi dell'Aquila

Not examinable · reading time about 90 minutes

Contents

Slide 1 — Lesson 3: Regression . . . 3

Part I — The model and its cost **3**
Slide 2 — Before we start . . . 3
Slide 3 — Today: the first real model . . . 3
Slide 4 — One dataset, all lesson . . . 4
Slide 5 — What we are trying to predict . . . 4
Slide 6 — What a linear model claims . . . 5
Slide 7 — Fitting means minimising something . . . 5
Slide 8 — Why squared, not absolute . . . 5
Worked example — the best single price for three houses . . . 6
Slide 9 — The cost function: mean squared error . . . 6
Slide 10 — Trying $\hat{y} = 2{,}400 \cdot \text{area} + 45{,}000$ . . . 6
Slide 11 — Squared error spends its attention on the worst house . . . 6
Slide 12 — A cost needs something to be compared against . . . 7
Question — what does the comparison say, and where does the 20,341 come from? . . . 7
Question — why does the last bullet say "R^2 depends on *this* test set's variance"? . . . 8
Worked example — the same model, two test sets . . . 8
Slide 13 — Fitting is finding where J is smallest . . . 9
Slide 14 — The cost is a bowl . . . 9

Part II — The exact solution **10**
Slide 15 — The picture behind the exact solution . . . 10
Question — does the figure mean there are only two features? And what is X? . . . 11
Slide 16 — θ and X: all parameters, all houses . . . 11
Slide 17 — The normal equation . . . 11
Slide 18 — The same equation, by hand . . . 12
Slide 19 — And it really is the minimum . . . 12
Worked example — one slope, no intercept, three houses . . . 13
Slide 20 — So why learn anything else? . . . 13
Slide 21 — How stretched is the valley? . . . 13
Worked example — two standardised features, rising correlation . . . 14

Part III — Gradient descent **14**

Slide 22 — The picture behind gradient descent . . . 14
Slide 23 — The gradients: one for w, one for b . . . 14
Worked example — three houses voting on the price of a square metre . . . 15
Slide 24 — One iteration, then the next . . . 15
Slide 25 — One step on two houses, from w = 0 and b = 0 . . . 16
Slide 26 — The path it takes . . . 16
Slide 27 — Three learning rates . . . 17
Worked example — a bowl of curvature 2 . . . 17
Slide 28 — Notebook 1, live . . . 17

Part IV — Curves, and the price of flexibility **18**
Slide 29 — Break . . . 18
Slide 30 — What if the relationship bends? . . . 18
Slide 31 — A relationship a line cannot follow . . . 18
Slide 32 — "Linear" means linear in the coefficients . . . 19
Worked example — a parabola by the normal equation . . . 19
Slide 33 — The straight line's residuals keep the shape . . . 20
Slide 34 — What "nothing left" looks like . . . 20
Slide 35 — So use more flexibility? . . . 21
Slide 36 — Root mean squared error (RMSE), by degree . . . 21
Slide 37 — The gap is the overfitting . . . 22
Slide 38 — Notebook 2, live . . . 22

Part V — Regularisation **22**
Slide 39 — Why flexibility goes wrong . . . 22
Slide 40 — Charge for size . . . 23
Slide 41 — Two rules that are not optional . . . 23
Slide 42 — What a penalty buys . . . 23
Slide 43 — The unpenalised fit leaves the picture . . . 24
Slide 44 — Too flexible, too rigid . . . 24
Slide 45 — Ridge has an exact solution too . . . 25
Worked example — one coefficient, four penalties . . . 25
Slide 46 — Why that one change fixes it . . . 25
Worked example — the same column twice . . . 26
Slide 47 — The picture behind Ridge vs Lasso . . . 26
Slide 48 — Ridge shrinks, Lasso selects . . . 27
Worked example — three coefficients, both penalties . . . 28
Slide 49 — The order Lasso drops features . . . 28
Question — what is the "budget", and where is it in the cost? . . . 28

Part VI — Reading coefficients **29**
Slide 50 — The case Ridge was invented for . . . 29
Worked example — one effect, two columns . . . 30
Slide 51 — The coefficients are noise . . . 30
Slide 52 — What a coefficient actually says . . . 30
Worked example — the same column, two coefficients . . . 31
Slide 53 — When can you trust a coefficient? . . . 31
Question — what does the note of slide 53 say? . . . 32

Question — ideally, should features be uncorrelated with each other and correlated with the label? . . . 32
Slide 54 — Notebook 3, live . . . 33
Slide 55 — What we did today . . . 33
Slide 56 — Homework: we discuss it on Friday 23 October . . . 33

What this document is. A companion to the slides of lesson 3: for every slide, what it means, the steps it leaves implicit, and the questions it tends to raise, answered under the slide they belong to. It is a second explanation of the same material, in a different voice from the handout, which remains the reference for the derivations. It is **not examinable**: nothing here is needed for the exam that is not already in the handout and the notebooks.

Slide numbers and titles are the ones in the lesson's slide deck, in `Slides/`. Where a section mentions "the notes", it means the speaker notes stored in the slide deck, which PowerPoint shows below each slide.

Worked examples sit under twelve slides (8, 12, 19, 21, 23, 27, 32, 45, 46, 48, 50, 52): a handful of invented values each, small enough to work through by hand. Their numbers are not the lesson's data; every one is computed by `commentary_examples.py`, beside this file, which also writes them here.

Slide 1 — Lesson 3: Regression

The first lesson in which a model is actually fitted. Lessons 1 and 2 built the frame: what learning from data means, why the data must be split before anything learns from it, and how to prepare data honestly inside a pipeline. From today the pipeline gets a model at its end.

Part I — The model and its cost

Slide 2 — Before we start

Exercise 2 is discussed first. The mistake most worth naming is fitting the scaler or the imputer before splitting the data: it is exactly what lesson 2 warned about, and seeing it in one's own work teaches more than any slide. The second bullet is the bridge: the `ColumnTransformer` built in lesson 2 is where every model of this course will now sit. We stop preparing data and start fitting models to prepared data.

Slide 3 — Today: the first real model

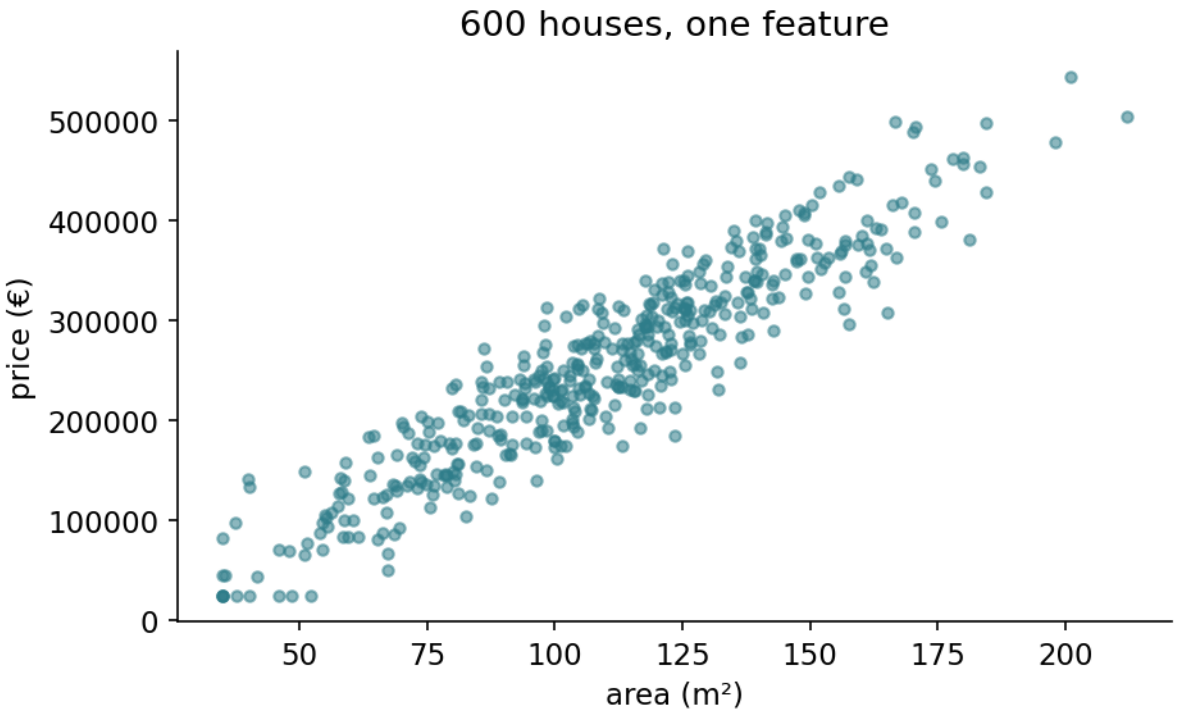
Regression comes first for three reasons that have nothing to do with it being simple. It has an **exact solution**, so we can see what fitting means without any iterative machinery in the way. It also has an **iterative solution**, gradient descent, which is the same algorithm that trains neural networks in lesson 9. And its **coefficients are readable** in the units of the problem: a linear model says what it thinks each feature is worth, in euros. That last property is why linear models

are still used in medicine and credit scoring, decades after more accurate methods appeared: when a decision must be explained to the person it affects, readability counts.

Slide 4 — One dataset, all lesson

600 houses, six measurements, a price; `train_test_split(X, y, test_size=0.25)` keeps 150 houses aside for testing and fits on 450. The data is **synthetic on purpose**: the prices were generated from known coefficients - 2,400 euros per square metre, 9,000 per bedroom, 14,000 per bathroom, minus 1,800 per year of age, minus 6,500 per kilometre from the centre, 12,000 for a garage - plus random noise with a standard deviation of 18,000 euros on each price. Because the truth is known, every estimate today can be checked against it, which no real dataset allows. Without the truth, "the coefficient is 7,503" is just a number; with it, "the coefficient is 7,503 and the truth is 9,000" is a lesson about when an estimate can be trusted (slides 52-53).

Slide 5 — What we are trying to predict



The dataset the whole lesson uses: 600 houses, price against floor area. The relationship is clearly there and clearly not exact, which is what makes it worth a model rather than a lookup table.

Before any model, look at the data. Two things are visible. The cloud slopes upwards, so a straight line is a reasonable first guess. And the spread around that slope is wide: that is the 18,000 euros of noise, which no model can remove. Try to estimate the slope by eye, in euros per square metre, before seeing the fit: if the guess lands near 2,400, the fitted number will mean something when it appears.

Slide 6 — What a linear model claims

The model is

$$\hat{y} = w_1 x_1 + w_2 x_2 + \cdots + w_n x_n + b,$$

one coefficient w_j per feature plus an intercept b. Read as a sentence: **each feature contributes a fixed amount per unit, and the contributions add up.** Every square metre adds the same 2,400 euros, whether it is the fortieth or the two-hundredth; every kilometre from the centre removes the same 6,500.

That is a strong claim, and often false: the tenth bedroom is not worth what the second was, and a garage is worth less in a suburb with free street parking. The right reaction is not to drop linear models but to know what has been assumed. Part IV shows how to check the assumption (the residual plots) and how to relax it (squared terms).

Slide 7 — Fitting means minimising something

To fit a model we need one number that says how badly a candidate model is doing, so that "fitting" becomes "make this number small". Adding up the errors fails at once: a house predicted 50,000 too high and one predicted 50,000 too low sum to zero, and the model would look perfect. So every error must count as a positive amount. Squaring does that; so does the absolute value. The next slide says why we square.

Slide 8 — Why squared, not absolute

Three reasons, in increasing order of depth.

- **Convenience.** The square is differentiable everywhere; the absolute value has a corner at zero, which complicates the optimisation.
- **A design decision.** Squaring makes one large error cost more than several small ones: being 40,000 euros wrong on one house costs as much as being 20,000 wrong on four ($40{,}000^2 = 4 \times 20{,}000^2$).
- **The deep reason.** If the noise is Gaussian, the coefficients that minimise the squared error are exactly the ones that make the observed prices most probable: the **maximum likelihood** estimate. Least squares is then not a convention but the natural answer.

The price of squaring: **squared error is sensitive to outliers**. One absurd price contributes its error squared and can drag the whole fit towards itself. That is why lesson 2's work on outliers is a prerequisite for this lesson, not a warm-up.

Worked example — the best single price for three houses

Three houses at 150, 200 and 700 thousand euros. Predict one number for all three: which number is best depends on how errors are charged.

guess	mean squared error	mean absolute error
200 (the median)	84167	**183.3**
350 (the mean)	**61667**	233.3

Squared error is minimised by the **mean**, 350; absolute error by the **median**, 200. One mansion drags the squared-error answer 150 thousand above the median, past two of the three houses — the outlier sensitivity on the slide, in one line. That is the price paid for differentiability and for the Gaussian story.

Slide 9 — The cost function: mean squared error

$$J(w,b) = \frac{1}{2m}\sum_{i=1}^{m}\left(\hat{y}^{(i)} - y^{(i)}\right)^2$$

m is the number of examples, $\hat{y}^{(i)} - y^{(i)}$ the error on example i. This is the **mean squared error (MSE)**, with an extra half in front. The half is there for convenience: differentiating a square brings down a factor 2, and the half cancels it. It cannot move the minimum, because multiplying a function by a constant does not change where its lowest point is. (scikit-learn's `mean_squared_error` has no half; slide 11 says so.)

Slide 10 — Trying $\hat{y}$ = 2,400 · area + 45,000

Three houses and one candidate model. Walk the first row: 80 square metres, the model says 2,400 × 80 + 45,000 = 237,000, the true price is 240,000, so the error $\hat{y} - y$ is −3,000 and its square is 9.0×10^6. The cost is $\frac{1}{2\times 3}(9.0 \times 10^6 + 1.69 \times 10^8 + 2.25 \times 10^8) \approx 6.7 \times 10^7$.

That number means nothing on its own. A cost is only meaningful **compared with another cost**, which is exactly what fitting does: it compares candidate models and keeps the cheapest.

Slide 11 — Squared error spends its attention on the worst house

Look at how the total is made. The 3,000-euro error contributes about **2%** of it; the two large errors contribute the other **98%**. Halving the worst error, the 15,000, cuts the cost by **42%**; halving the best one, the 3,000, cuts it by only 1.7%. Same effort, twenty-five times the reward: the fit will therefore bend towards its worst predictions and largely ignore the ones it already gets nearly right.

Most of the time that is what you want. The flip side is lesson 2 again: one mis-recorded price contributes its error squared and pulls the whole fit towards itself. Minimising the absolute error instead would follow the bulk of the data and pay much less attention to the outlier, which is what robust regression (a family of methods built to resist outliers) does.

In code, `mean_squared_error(y, y_pred)` returns the mean of the squared errors, without the half of slide 9.

Slide 12 — A cost needs something to be compared against

Lesson 1's habit, applied to regression for the first time: before a score means anything, compare it with what a trivial answer scores. For a classifier the trivial answer was always the majority class; for a regression it is **always predicting the mean**, `DummyRegressor()`, ignoring every input.

The comparison uses the **root mean squared error**, RMSE = $\sqrt{\text{MSE}}$: the square root brings the number back to the units of the target, euros here. On the 150 test houses:

Model	RMSE
always predict the mean	96,440 €
the fitted six-feature model	20,341 €

The same comparison written as one number is the **coefficient of determination**,

$$R^2 = 1 - \frac{\sum_i (y_i - \hat{y}_i)^2}{\sum_i (y_i - \bar{y})^2} = 0.956 \quad (\texttt{r2_score}).$$

The numerator is the squared error the model still makes, the denominator the squared error that predicting the mean $\bar{y}$ would make. So 1 is perfect, 0 means no better than the mean, and a negative value means worse than the mean, which does happen on a test set.

Question — what does the comparison say, and where does the 20,341 come from?

> *Slide 12: "baseline RMSE €96,440; model RMSE €20,341, so the model is doing much better than the trivial answer". Explain it better.*
>
> *The €20,341 comes from a trained model, but the training is not shown. Can I say that the training comes later, and that this number only serves as a comparison with the one obtained with the mean?*

Yes, that is the right way to read it. At slide 12 no model has been trained yet: the 20,341 is a **preview**, the result of the linear regression that slides 15-27 will build, first with the exact solution and then with gradient descent, and that notebook 1 computes. What matters now is the comparison:

- predicting the mean price for every house, which learns nothing from the features, gives an RMSE of 96,440 euros;
- the model using all six features gives 20,341 euros, about 4.7 times smaller.

So the features carry real information about the price, and the model is using it. Note also what the comparison is *not*: it is not against perfection. An RMSE of zero is impossible here, because the prices contain 18,000 euros of noise that no model can predict.

Two details. The model is fitted on the 450 training houses and the RMSE is computed on the 150 test houses, which the fit never saw. And the RMSE is not "the average error per house": squaring

before averaging gives large errors more weight, so the RMSE is larger than the average absolute error.

Question — why does the last bullet say "R² depends on *this* test set's variance"?

> *Slide 12: why does the last bullet say "Quote the euros too: R^2 depends on this test set's variance"?*

Look at the denominator of R^2: $\sum_i (y_i - \bar{y})^2$ is m times the variance of the true prices **in the test set used**. So R^2 measures the model against how spread out that particular set of prices is. If the prices are very different from one another, predicting the mean is a poor strategy and R^2 comes out high; if they are all similar, the mean is already a good guess and the same model gets a low R^2.

The handout measures it on the lesson's own data. Keep the fitted model exactly as it is and score it on the 55 test houses whose price lies within half a standard deviation of the mean, houses that resemble one another:

Test set	RMSE	R^2
all 150 houses	20,341 €	0.956
the 55 similar ones	18,661 €	0.448

The model is unchanged and its error in euros **improved**, yet R^2 fell from 0.956 to 0.448. Nothing contradicts anything: the mean simply became much harder to beat. Two consequences: R^2 cannot compare models measured on different test sets, and a report should always give the RMSE in euros beside it, because the euros say how far off the model is and R^2 only says how much better than the mean.

Worked example — the same model, two test sets

Four houses (k EUR), and the model's predictions:

test set	prices	predictions	RMSE, model	RMSE, predict the mean	R^2
varied	200, 250, 300, 450	210, 240, 320, 430	15.8	93.5	**0.971**
look-alike	290, 300, 310, 300	295, 305, 305, 300	4.3	7.1	**0.625**

$R^2 = 1 - \mathrm{SS}_{\mathrm{res}}/\mathrm{SS}_{\mathrm{tot}}$: on the varied set $1 - 1000/35000$, on the look-alikes $1 - 75/200$. The model's error **fell** from 15.8 to 4.3, and yet R^2 fell too — because the look-alike houses leave the mean-predicting baseline almost nothing to get wrong. R^2 is a ratio to the test set's own spread; RMSE is in thousands of euros.

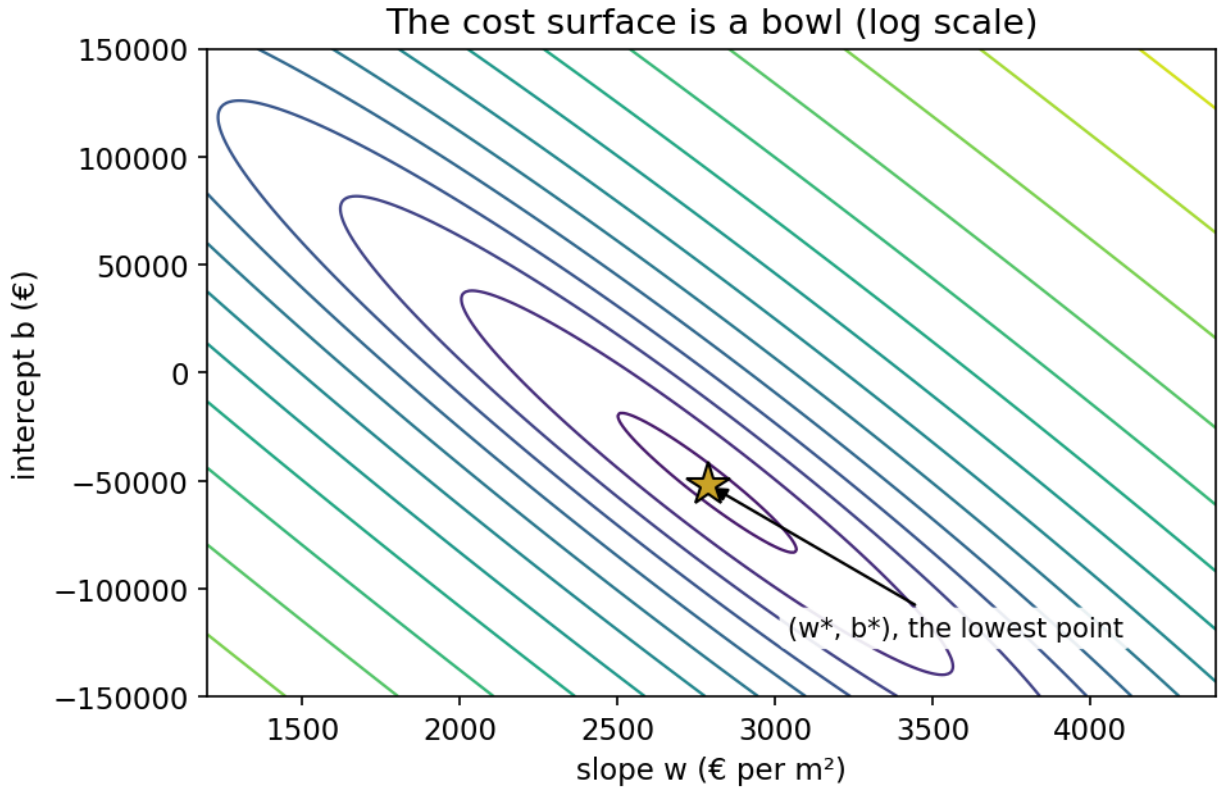
Slide 13 — Fitting is finding where J is smallest

$$(w^*, b^*) = \arg\min_{w,\,b} \; J(w, b)$$

This is the whole of "fitting" in one line. w^* and b^* are the slope and the intercept at which the cost J is smallest; the star is only a label meaning "the best values". $\arg\min$ is the notation of lesson 1, where the trained model was the arg min of the training error: it gives back the point where the minimum is reached, not the minimum itself. So the answer to this formula is a pair of numbers, a slope and an intercept, and not a cost.

The rest of the first half of the lesson is two ways of computing that pair: exactly, with the normal equation (slides 15-19), and step by step, with gradient descent (slides 22-27). The next slide draws J as a surface over (w, b); the pair asked for here is its lowest point.

Slide 14 — The cost is a bowl



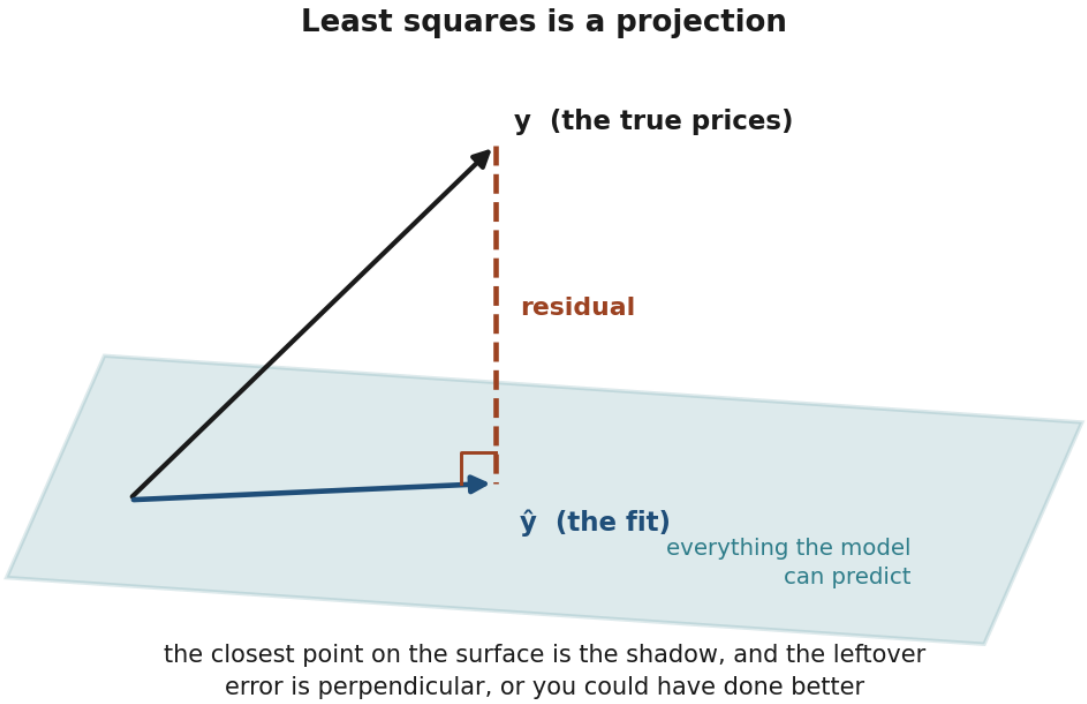
The cost as a function of the two parameters. It is a bowl, and a bowl has exactly one bottom — which is why squared error can be minimised exactly rather than searched for. The gold star is that bottom, (w^*, b^*): the grid point with the lowest cost, a slope of 2,790 and an intercept of −52,000, within one grid step of the exact answer, computed in section 3.

With two parameters, slope and intercept, the cost is a surface over the plane of their values, and fitting means finding its lowest point. The important structural fact is that this bowl is **convex**: it has exactly one minimum and no smaller dips where a search could get stuck. Most methods later

in the course do not have this property: the cost surfaces of lesson 9's networks have many local minima, and much of the difficulty of training them comes from that.

Part II — The exact solution

Slide 15 — The picture behind the exact solution



Least squares as a shadow. The fitted values are the projection of y onto the space the columns of X can reach, and the residual is what is left over — perpendicular to that space, which is exactly what $X^\top(X\theta - y) = 0$ says.

The intuition before the algebra. Collect the m true prices into one vector y, a point in an m-dimensional space. Each feature column is also such a vector, and so is the column of ones that carries the intercept. Every prediction the model can make, $\hat{y} = X\theta$, is a combination of those columns, so all possible predictions lie on a flat surface (a plane, in the picture) spanned by the columns. The true prices almost certainly do not lie on it: no combination of area and age reproduces them exactly.

So the best the model can do is the point of the surface **closest** to y: its shadow, the perpendicular projection. Why perpendicular? If the leftover error, the residual, had any component lying *along* the surface, the model could have moved in that direction and done better. At the optimum the residual must be at right angles to every column - which is exactly what the normal equation of slide 17 says, and where their name comes from (normal means perpendicular).

Question — does the figure mean there are only two features? And what is X?

> *Slide 15: so in the example of the figure $\hat{y}$ would live in a two-dimensional space? Does it mean there are only two features?*
>
> *Slide 15: remind me what the matrix X contains in its rows and its columns.*

The matrix first. X is the **design matrix**: **one row per example** (a house) and **one column per feature**, plus a first column of ones for the intercept. With m houses and n features it is $m \times (n+1)$; for the whole lesson's data, 600 houses and six features, 600×7. The parameter vector is $\theta = (b, w_1, \dots, w_n)$, and all m predictions at once are $\hat{y} = X\theta$. With the three houses of slide 18, using area only:

$$X = \begin{pmatrix} 1 & 80 \\ 1 & 120 \\ 1 & 200 \end{pmatrix}, \qquad \hat{y} = X \begin{pmatrix} b \\ w \end{pmatrix} = b \begin{pmatrix} 1 \\ 1 \\ 1 \end{pmatrix} + w \begin{pmatrix} 80 \\ 120 \\ 200 \end{pmatrix}.$$

Now the figure. Here the price vector y has three entries, one per house, so it lives in a three-dimensional space. The predictions are combinations of **two** vectors, the column of ones and the column of areas, so they fill a two-dimensional plane inside that space: the plane in the figure. So the plane's dimension is the **number of columns of X**, which is the number of features plus one for the intercept - here one feature, area, plus the intercept. It is not the number of houses, and it is not "two features".

In general, the predictions form a flat subspace whose dimension is the number of independent columns of X (its rank), at most $n + 1$. With the lesson's 600 houses and six features, that is a seven-dimensional subspace inside a 600-dimensional space. Nobody can draw it, but the geometry is the same: the fit is the point of that subspace closest to y.

Slide 16 — θ and X: all parameters, all houses

The two symbols the normal equation is written in. θ is every parameter of the model stacked in one column, the intercept first:

$$\theta = \begin{bmatrix} b \\ w_1 \\ \vdots \\ w_n \end{bmatrix}$$

X is the table of features with a column of ones in front: one row per house, then one column per feature. The ones are there for b. Multiply one row of X by θ and the 1 picks up b while each feature picks up its own w, which gives $b + w_1 x_1 + \cdots + w_n x_n$, the linear model of slide 6. So $X\theta$ is that prediction for every house at once, and the cost can be written as a function of θ alone, $J(\theta)$. The question under slide 15 works this through on the three houses.

Slide 17 — The normal equation

$$\theta = (X^\top X)^{-1} X^\top y$$

The perpendicularity of slide 15 written in symbols: the residual $y - X\theta$ must be perpendicular to every column of X, that is $X^\top(y - X\theta) = 0$, which rearranges to $X^\top X\,\theta = X^\top y$ and, when $X^\top X$ can be inverted, to the formula above. Four lines of NumPy, and notebook 1 shows them agreeing with scikit-learn's `LinearRegression` to 1.82×10^{-12} euros per square metre. The handout, section 3.1, derives it by differentiating the cost and setting the gradient to zero.

Slide 18 — The same equation, by hand

The whole method on three houses: area 80, 120 and 200 m², prices 240k, 320k and 540k (prices in thousands, to keep the numbers readable). X has a column of ones for b beside the areas, and $\theta = (b, w)$.

- $X^\top X$: the top-left entry counts the houses, 3; the off-diagonal entries are $80+120+200 = 400$; the bottom-right is $80^2 + 120^2 + 200^2 = 60{,}800$.
- $X^\top y$: $240 + 320 + 540 = 1{,}100$, and $80 \cdot 240 + 120 \cdot 320 + 200 \cdot 540 = 19{,}200 + 38{,}400 + 108{,}000 = 165{,}600$. This second dot product is where an arithmetic slip most easily hides.
- Determinant: $3 \times 60{,}800 - 400^2 = 22{,}400$; invert, multiply, and out comes a slope of about **2,536 euros per square metre** and an intercept of about 28,600 euros.

A slope within 6% of the true 2,400, from three houses. Do not read that as "more data does better": the same one-feature fit on 450 houses gives **2,785**, 16% too high. More rows cannot fix it because nothing is broken. Asked what a square metre is worth *when area is all you are told*, the honest answer includes the bedrooms and bathrooms that come with larger houses. With all six features the area coefficient drops to 2,421. What a coefficient means depends on what else is in the model, which is Part VI.

Slide 19 — And it really is the minimum

Setting the gradient to zero finds a flat spot; on its own it does not say whether the flat spot is the bottom, a ridge or a small dip. Here it is the bottom, because the second derivative of the cost is $X^\top X$ (divided by m), and $X^\top X$ is never negative in any direction: for any direction v, $v^\top X^\top X v = \|Xv\|^2$, a squared length, which cannot be negative. So the cost is **convex**, and any stationary point (a point where the gradient is zero) is *the* minimum, not *a* minimum.

This is a luxury. In lesson 9 nobody can promise that the point training reached is the point we wanted; today we can. In code, `LinearRegression().fit(X, y)` solves the same least-squares problem.

Worked example — one slope, no intercept, three houses

Areas 1, 2, 3 (hundreds of m²), prices 2, 3, 6 (hundreds of k EUR), model $\hat{y} = wx$. Setting the derivative to zero gives $w^* = \sum xy / \sum x^2 = 26/14 = 1.857$. The cost around it:

w	$J(w)$
1.257	0.9590
1.557	0.3290
1.857	0.1190
2.157	0.3290
2.457	0.9590

Step 0.3 either side and the cost rises by exactly the same amount, $0.2100 = \frac{1}{2} \cdot \frac{\sum x^2}{m} \cdot 0.3^2$: the cost is a parabola whose curvature $\sum x^2/m = 4.667$ can never be negative. That is the one-feature case of $\|Xv\|^2 \geq 0$ — so the flat spot is the bottom.

Slide 20 — So why learn anything else?

Three ways the exact solution fails.

- **Size.** Inverting an $n \times n$ matrix costs about n^3 operations: a billion at a thousand features, out of reach at a hundred thousand. Every large model in this course is trained iteratively.
- **Two columns carrying one fact.** The same measurement in metres and in feet: then $X^\top X$ has no inverse at all.
- **Nearly redundant columns**, the dangerous case because nothing visibly breaks. The inverse exists but is enormous, and small changes in the data produce large changes in the coefficients: notebook 3 shows an area coefficient moving from 9,261 to 45,307 across four random splits of the same data, with an invertible matrix every time.

Slide 21 — How stretched is the valley?

The **condition number** κ is the steepest curvature of the cost bowl divided by the shallowest: how many times steeper the steepest direction is than the flattest. A round bowl has $\kappa = 1$; a long narrow ravine has a large κ. On the housing data:

Design matrix	Condition number
the six features as recorded	285
the same six, standardised	3.4
standardised, plus `area_sqft` beside `area_sqm`	2,286

Why it matters twice. For the exact solution, a stretched valley is what makes the inverse enormous and the answer unstable: along the flat direction, moving one coefficient up and the other down changes the predictions hardly at all, so the data cannot decide between them. For gradient descent, one step size has to serve every direction: the steep direction sets the limit, and the shallow one

crawls. The handout (section 4.4) shows that the number of iterations grows roughly in proportion to κ, so standardising here saves a factor of about 80 in iterations.

Worked example — two standardised features, rising correlation

Two features, each standardised, correlated r. The curvatures of the cost are the eigenvalues of $\begin{pmatrix} 1 & r \\ r & 1 \end{pmatrix}$, which are $1 + r$ and $1 - r$, so $\kappa = (1 + r)/(1 - r)$:

r	steepest	shallowest	κ
0.0	1.00	1.00	**1**
0.5	1.50	0.50	**3**
0.9	1.90	0.10	**19**
0.99	1.99	0.01	**199**

Standardising fixes the units, not the correlation: two features that nearly say the same thing still make a stretched valley. And gradient descent needs roughly κ steps, so at $r = 0.99$ about two hundred times as many as with independent features.

Part III — Gradient descent

Slide 22 — The picture behind gradient descent

You are on a hillside in fog: you cannot see the valley, but you can feel which way the ground slopes under your feet. So you take a step downhill, feel again, and repeat until the ground is flat. That is the whole algorithm. The gradient points in the direction of steepest *ascent*, so the step goes against it, and the **learning rate** α sets the length of the stride. The danger is just as intuitive: stride too far and you cross the valley and land higher on the opposite slope.

Slide 23 — The gradients: one for w, one for b

$$\frac{\partial J}{\partial w} = \frac{1}{m} \sum_i \left(\hat{y}^{(i)} - y^{(i)} \right) x^{(i)}, \qquad \frac{\partial J}{\partial b} = \frac{1}{m} \sum_i \left(\hat{y}^{(i)} - y^{(i)} \right)$$

Two parameters, two gradients: how fast the cost rises as w grows, and how fast it rises as b grows.

With several features the same holds for every coefficient. For the coefficient w_j, the step is

$$w_j \leftarrow w_j - \alpha \, \frac{\partial J}{\partial w_j}.$$

Writing the error of example i as

$$e^{(i)} = \hat{y}^{(i)} - y^{(i)},$$

the two gradients become

$$\boxed{\frac{\partial J}{\partial w_j} = \frac{1}{m}\sum_i e^{(i)} x_j^{(i)}} \qquad \text{and, for the intercept,} \qquad \boxed{\frac{\partial J}{\partial b} = \frac{1}{m}\sum_i e^{(i)}}$$

With one feature, x_j is just the area x, and these are the two formulas on the slide. Read the first gradient as a sentence: each example pulls coefficient w_j in proportion to **two things** - how wrong its prediction was, and how large feature j was for that example. The gradient is a **weighted vote**, and the weights are the feature values: a badly mispriced 200 m² house moves the area coefficient much more than a 40 m² house with the same error. That is why unscaled features cause trouble, lesson 2's argument from a different direction. The intercept's gradient has no x in it: every example gets an equal vote there, as if b were the coefficient of a feature that is always 1.

Worked example — three houses voting on the price of a square metre

The gradient for the area coefficient is the average of (error × area):

area (m²)	error $\hat{y} - y$ (k EUR)	pull = error × area
80	+10	+800
120	-5	-600
200	-20	-4,000

$\partial J/\partial w = (+800 - 600 - 4,000)/3 = -1,266.7$: negative, so the update **raises** w — the model was charging too little per m². The big house supplies 74% of the total pull: it was the most wrong *and* the largest. A one-room flat with the same error would barely be heard.

Slide 24 — One iteration, then the next

Gradient descent is a loop, and each round does the same five things in order:

1. **Predict** every house with the current w and b: $\hat{y}^{(i)} = w\,x^{(i)} + b$.
2. **Measure** every error: $e^{(i)} = \hat{y}^{(i)} - y^{(i)}$.
3. **Average** the errors into the two gradients of slide 23, $\partial J/\partial w = \frac{1}{m}\sum_i e^{(i)} x^{(i)}$ and $\partial J/\partial b = \frac{1}{m}\sum_i e^{(i)}$.
4. **Step** both parameters against their gradients, by the same learning rate: $w \leftarrow w - \alpha\,\partial J/\partial w$ and $b \leftarrow b - \alpha\,\partial J/\partial b$.
5. **Go back to 1.**

Step 5 is what makes it iterative. After step 4 the model is a different line, so every prediction changes, so every error changes, and the gradients have to be computed again from the new w and b: nothing from the previous round is reused except the two parameters themselves. Both updates in step 4 use the gradients of the *old* values, before either parameter has moved. The first round starts from a guess, $w = 0$ and $b = 0$ here, and the loop stops after a fixed number of rounds or when the cost stops falling. Collecting b and w into θ, steps 3 and 4 are the handout's $\theta \leftarrow \theta - \alpha\nabla J(\theta)$ in one line.

In notebook 1 the loop is the `gradient_descent` function, and its body is these steps line for line: `error = (w * x + b) - y`, then `grad_w = (error * x).mean()` and `grad_b = error.mean()`, then the two updates. The next slide is one round of it, done by hand on two houses.

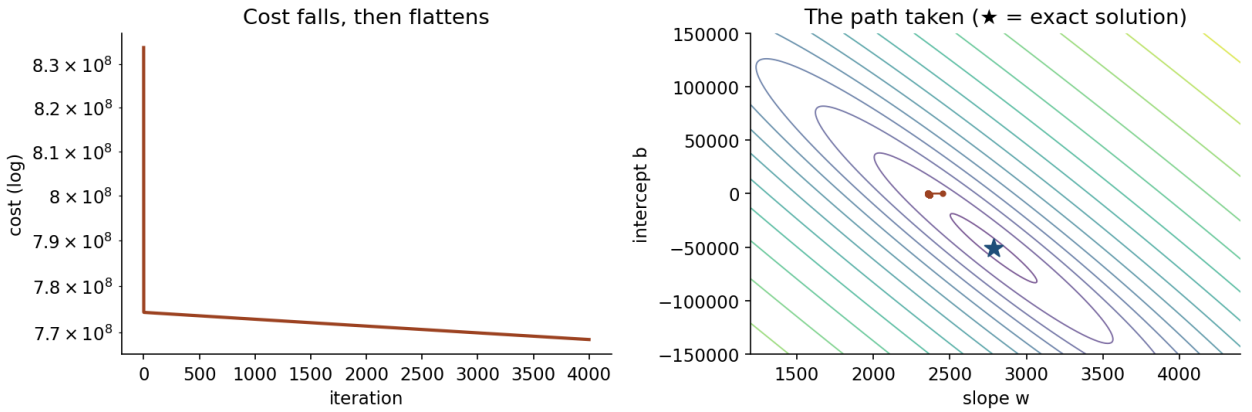
Slide 25 — One step on two houses, from w = 0 and b = 0

Two houses, 80 m² at 240k and 200 m² at 540k, a start at $w = 0$, $b = 0$, and $\alpha = 10^{-5}$. Both predictions are 0, so the errors are −240 and −540 (thousands):

- slope gradient: ½[(−240)(80) + (−540)(200)] = −63,600, so w becomes 0.636;
- intercept gradient: $\frac{1}{2}(-240 - 540) = -390$, so b becomes 0.0039.

Both gradients are negative because the model predicts zero, far too low, so both parameters increase. The thing to notice: after one identical step the slope has moved **163 times further** than the intercept, only because areas are around 140 while the intercept's "feature" is always 1. With one shared learning rate the intercept crawls long after the slope has arrived - notebook 1 shows it still creeping after 4,000 iterations. That is the concrete cost of unscaled features.

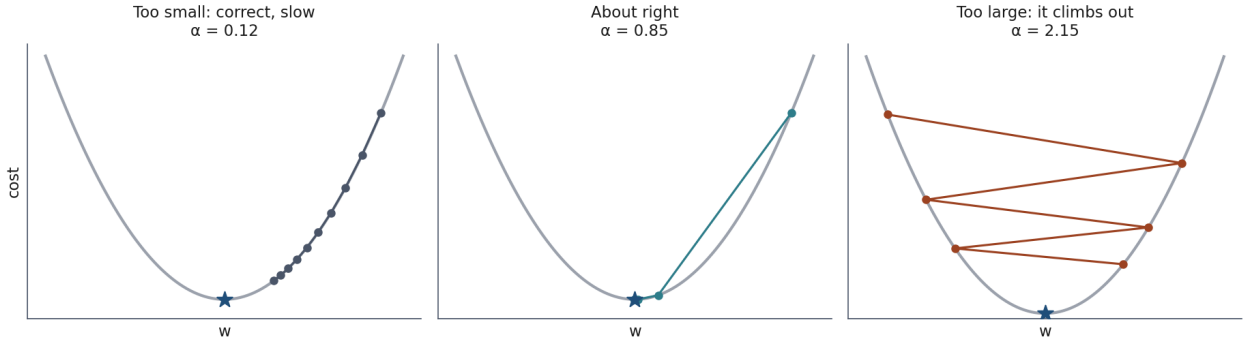
Slide 26 — The path it takes



The path taken across the cost surface. Each step is against the gradient, and the steps shorten as the slope flattens near the bottom.

The same run seen two ways. On the left, the cost against the iteration: it drops steeply in the first few dozen steps and then flattens into a long tail still creeping at 4,000. The model was essentially fitted early; most iterations bought almost nothing. On the right, the run drawn on the contour lines of the cost, with the star at the exact solution: the path arrives quickly in the steep direction, then crawls along the floor of the valley towards the star. That crawl is the 163-to-1 asymmetry of slide 25, and the condition number of slide 21, made visible.

Slide 27 — Three learning rates



Three learning rates on the same problem: too small and it crawls, about right and it arrives, too large and it climbs the opposite wall. The third case diverges — the cost goes up.

Too small: correct but slow, the cost falls at every step and simply needs too many of them. About right: it arrives. Too large: each step overshoots the minimum and lands higher on the far side, the next overshoots further, and the cost explodes, often to `nan` within a few dozen iterations.

The boundary is not folklore. For a bowl of curvature c, each step multiplies the distance from the minimum by $(1 - \alpha c)$, so gradient descent converges exactly when $0 < \alpha < 2/c$ (handout, section 4.3). With several features the condition must hold in every direction, and the binding one is the steepest, set by the feature with the largest variance: **the safe learning rate is set by the worst-scaled feature**, one more reason to scale them all. A practical recipe: start at 0.01 on scaled features, watch the cost, and divide by three if it rises. Lesson 5 replaces the recipe with a search.

Worked example — a bowl of curvature 2

$J(w) = \frac{1}{2} \cdot 2\, w^2$, minimum at 0, start at $w = 1$. The gradient is $2w$, so each step multiplies w by $1 - 2\alpha$:

α		each step	first three steps
0.1	small	$\times(0.8)$	0.800, 0.640, 0.512
0.5	exactly 1/c	$\times(0.0)$	0.000, 0.000, 0.000
0.9	large	$\times(-0.8)$	-0.800, 0.640, -0.512
1.1	too large	$\times(-1.2)$	-1.200, 1.440, -1.728

Below $\alpha = 1/c$ the path creeps in from one side; between $1/c$ and $2/c$ it overshoots and oscillates but still closes in; past $2/c = 1$ every step lands further out than the last. With several features the steepest direction sets c — which is why one badly scaled feature limits the step for all of them.

Slide 28 — Notebook 1, live

The table on the slide maps each section of the notebook to the slides it comes from, the calls it uses and the number to come back with. Two moments are worth pausing on: the agreement between a four-line normal equation and scikit-learn to twelve decimal places, and the coefficient table at the

end, which sets up Part VI. If someone's gradient descent diverges, that is a good thing to happen: it is slide 27 on their own screen.

Part IV — Curves, and the price of flexibility

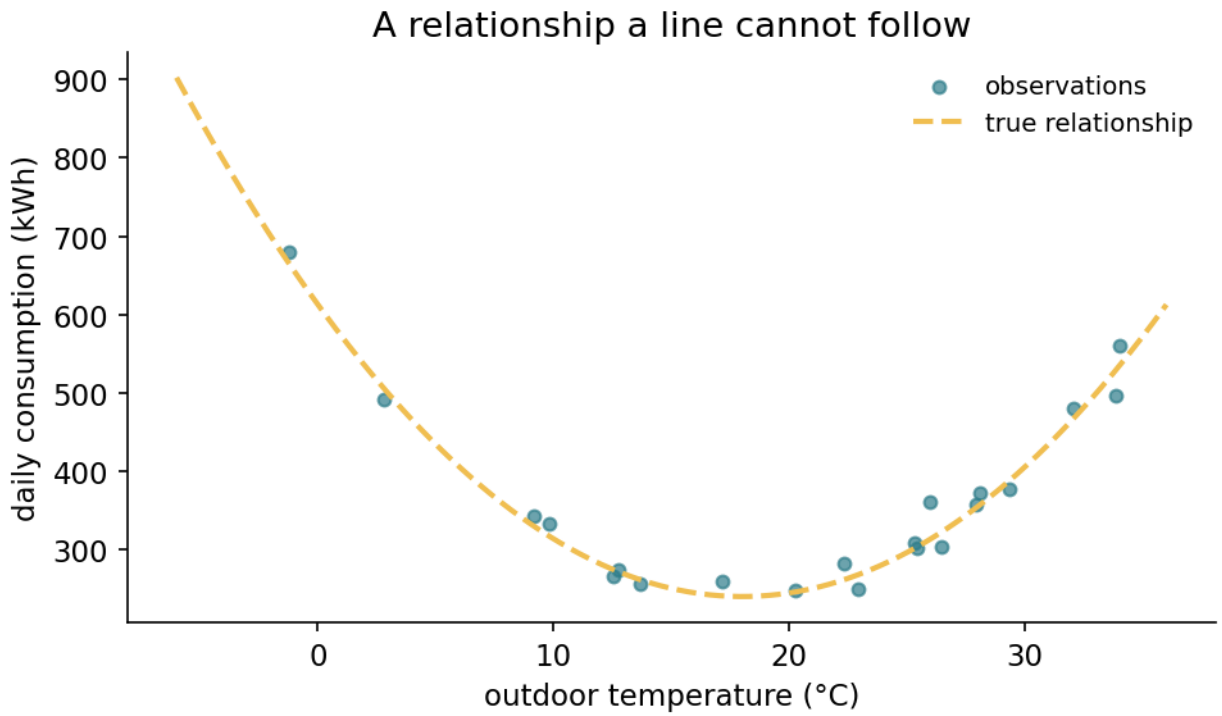
Slide 29 — Break

The second half takes the same machinery to relationships that are not straight lines, and to what happens when a model is given too much freedom.

Slide 30 — What if the relationship bends?

Daily energy consumption against outdoor temperature: high when it is cold (heating), high when it is hot (cooling), lowest somewhere around 18 °C. A straight line would average through the middle: too high in the centre and too low at both ends, wrong in a structural way rather than slightly imprecise. In code, `PolynomialFeatures(degree=2)` adds the squared temperature, t^2, as a second column, which is the trick of slide 32.

Slide 31 — A relationship a line cannot follow



Daily energy against outdoor temperature, with a minimum near 18 °C. Heating below, cooling above: no straight line can follow this, and no amount of fitting will make one.

The U is obvious to a human eye in under a second; the hard part was never seeing the shape but getting a model to represent it. Note the sample size: 30 days of measurements, of which 21 are used for training. Deliberately few, because the overfitting about to appear needs a model with nearly as many coefficients as observations - a situation far more common in real work than people expect.

Slide 32 — "Linear" means linear in the coefficients

$$\hat{y} = w_1 t + w_2 t^2 + b$$

The surprise: this is still a linear model. "Linear" refers to the **coefficients**, not to the inputs: the prediction is a weighted sum of columns, and nothing in the derivation of slides 15-19 required those columns to be straight functions of the temperature. So it is the same least squares, the same normal equation and the same code, on a design matrix with one extra column, t^2. The same trick covers interactions such as $x_1 x_2$, logarithms, anything that can be computed from the inputs.

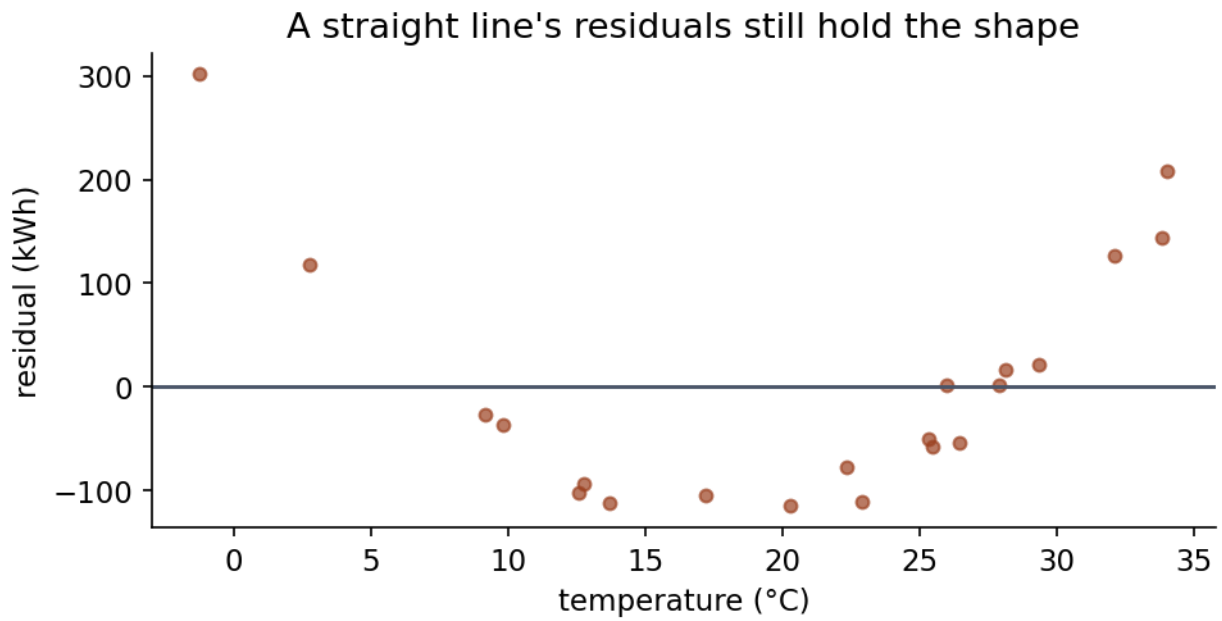
Worked example — a parabola by the normal equation

Three days: temperature $t = -1$, 0, 1 (standardised), energy 3, 1, 3. Give the design matrix a column of t^2:

$$X = \begin{pmatrix} 1 & -1 & 1 \\ 1 & 0 & 0 \\ 1 & 1 & 1 \end{pmatrix}, \qquad \theta = (X^\top X)^{-1} X^\top y = (1, 0, 2)$$

so $\hat{y} = 1 + 2\,t^2$ — the U, through all three points. Nothing in the method changed: same equation, same code, one more column. The curve is in the **columns**; the model is still a straight line in $(1, t, t^2)$.

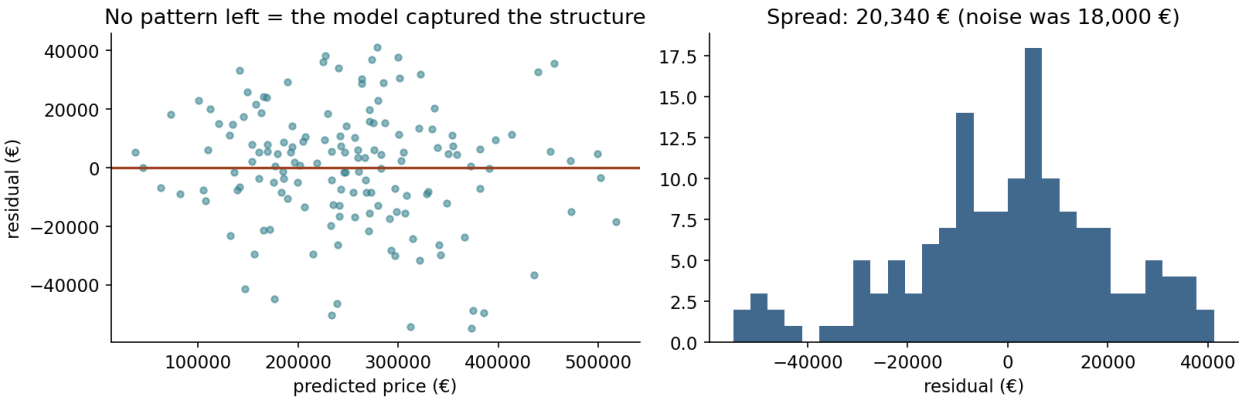
Slide 33 — The straight line's residuals keep the shape



The straight line's residuals still carry the shape of the curve. Residuals that hold a pattern are the model telling you it has not finished.

The diagnostic to learn. Plot the residuals, true value minus prediction, against the feature. Here they still trace the U: there is structure in the data that the straight line did not capture. A pattern in the residuals means the model has missed something.

Slide 34 — What "nothing left" looks like

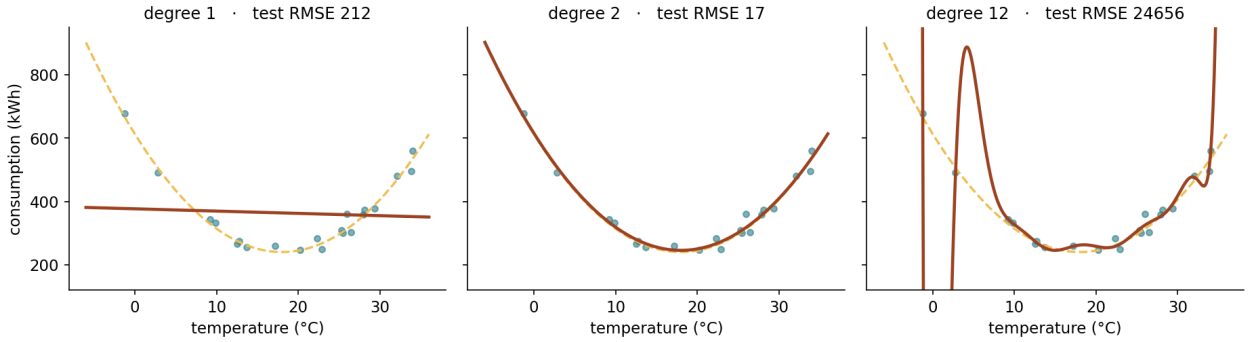


The right degree leaves residuals with no pattern left in them — scattered around zero, no shape. That is what "nothing left to model" looks like, and it is the check to run on any fit.

The contrast, from notebook 1's housing fit. A formless cloud centred on zero, with no pattern as you sweep from left to right, and a spread matching the 18,000 euros of noise that was put in. That is the best possible outcome: the model has taken everything there was to take, and what remains

is what nobody could predict. A U means structure missed; a cloud means done. One line of code after every fit, every time.

Slide 35 — So use more flexibility?



The same 21 observations at three degrees. The rightmost passes closest to the training points, which is precisely why it is the worst model here.

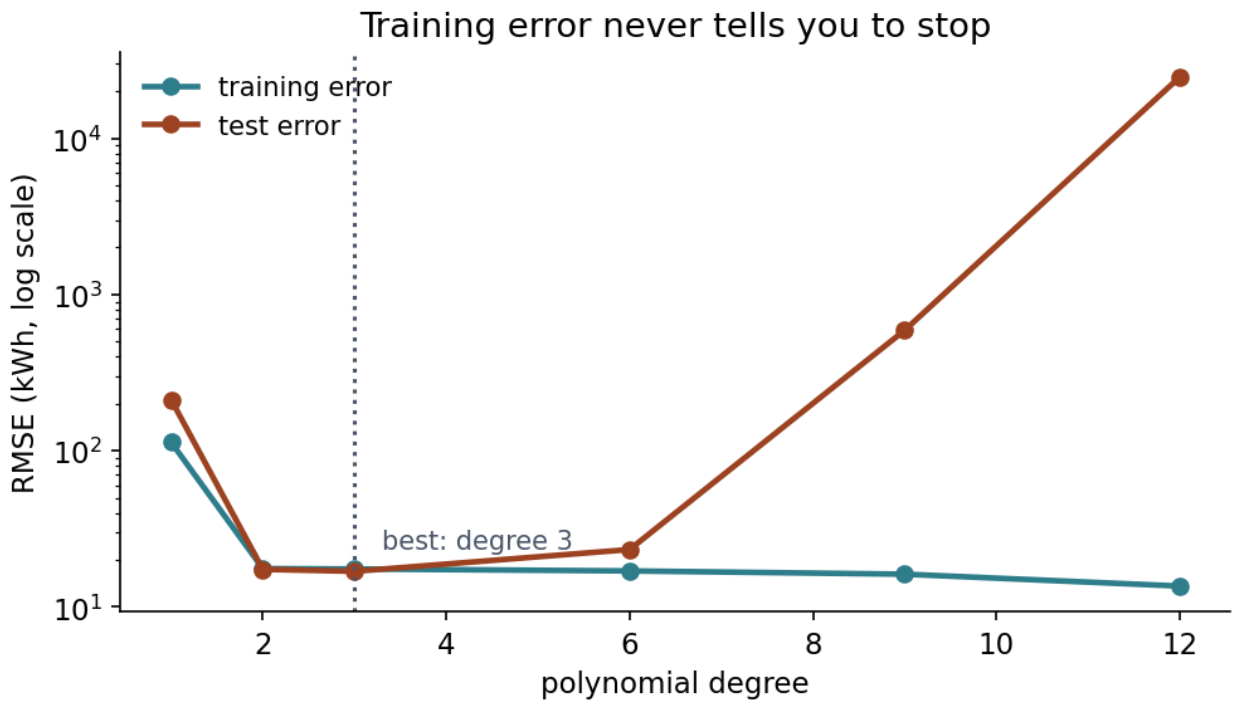
Degree 1 is too rigid, degree 2 about right, degree 12 wild. Two questions to ask yourself. Which would you pick if the dashed true curve were not drawn? That is the real situation: you only ever see the points. And which panel passes closest to the training points? The right one, degree 12. Choosing by training error would pick it every time.

Slide 36 — Root mean squared error (RMSE), by degree

Degree	Train RMSE (kWh)	Test RMSE (kWh)
1	113.3	212.1
3	17.5	**16.9**
9	16.2	590.1
12	13.6	24,655.7

The RMSE is in kilowatt-hours, the units of the target, so it can be compared with the consumption itself. Read the columns against each other. The training error falls at every step, never once suggesting that anything is wrong. The test error is lowest at degree 3 and then climbs: 590 at degree 9, 24,656 at degree 12. The model that fits the training data best is more than a thousand times worse on data it has not seen. With 21 training points, a degree-12 polynomial has 12 coefficients plus an intercept: more than one parameter for every two observations. This is lesson 1's gap between empirical and expected risk, with numbers.

Slide 37 — The gap is the overfitting



Training error falls with every degree added; test error turns around. The gap between the two curves is the overfitting, and the turning point is what regularisation exists to find without hunting for it by hand.

The table drawn. The training curve falls and never turns up: if it were the only curve you could see - and on your own data, without a test set, it is - you would keep adding degrees forever. The test curve turns. The widening gap between the two is what **overfitting** means. Lesson 5 is about measuring the upper curve honestly when there is no test set to spend.

Slide 38 — Notebook 2, live

The table maps the notebook's sections to the slides and calls. Run the degree sweep and stop at the table: predict which degree wins before scrolling. Most people guess too high, because the instinct that more flexibility is better is exactly what the lesson is dismantling. The coefficient printout at the end is the number the next slide is about.

Part V — Regularisation

Slide 39 — Why flexibility goes wrong

Largest coefficient at degree 2: **411**. At degree 12: **3,097,038,010**. The huge coefficients work in **opposition**: with 21 points and 12 coefficients there are many ways to pass close to every point, and least squares finds one whose terms nearly cancel - one pushing the curve up where the next

pushes it down, the cancellation failing exactly where a training point sits. Between the points nothing holds the swing back. Which suggests the question for the rest of the lesson: if enormous opposing coefficients are the symptom, what if coefficient size had a price?

Slide 40 — Charge for size

$$J_{\text{ridge}} = \text{MSE} + \lambda \sum_j w_j^2 \qquad J_{\text{lasso}} = \text{MSE} + \lambda \sum_j |w_j|$$

Two penalties, one difference: **Ridge** charges the sum of the squared coefficients, **Lasso** the sum of their absolute values. The **regularisation strength** λ sets the exchange rate between fitting the data and keeping coefficients small; at $\lambda = 0$ both are ordinary least squares. A naming trap: this course writes the penalty as λ and keeps α for the learning rate, but scikit-learn calls the penalty `alpha`, so `Ridge(alpha=0.01)` in code is $\lambda = 0.01$ here.

Slide 41 — Two rules that are not optional

Both produce results that look fine when broken, which is why they get a slide.

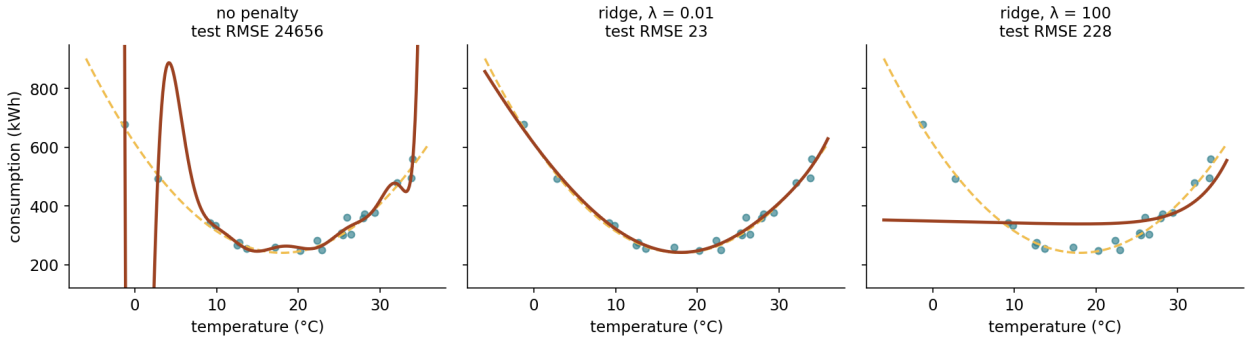
- **The intercept is never penalised.** It says where the surface sits, not what any feature is worth; shrinking it would drag every prediction towards zero, a meaningless place for house prices. scikit-learn's `Ridge` and `Lasso` leave it out for you.
- **Features must be scaled first.** The penalty is a sum over coefficients, so the same feature measured in metres and in kilometres attracts penalties a thousand times apart: whichever feature happens to be recorded in small units needs a large coefficient and is punished hardest, by an accident of units. In lesson 2 scaling was about speed and could be skipped at a cost; here it is about correctness. In code: `make_pipeline(StandardScaler(), Ridge(alpha=0.01))`.

Slide 42 — What a penalty buys

Model	Train RMSE	Test RMSE	max |w|
none	13.6	24,655.7	3,097,038,010
= 0.01	18.0	**22.8**	365
= 1	44.5	105.2	156
= 100	96.0	227.6	6

Read the last column first: three billion down to **365**, from a penalty of one hundredth - the number this lesson leaves behind. The test error falls from 24,656 to 22.8, near the noise floor. Then the training column, the honest part: it gets **worse** at every step, 13.6, 18, 45, 96. That is the trade made deliberately, a worse fit on the data we have in exchange for a better fit on data we do not. And it stops paying: at $\lambda = 1$ the test error is 105, at $\lambda = 100$ it is 228, because the model has become too rigid to follow the curve. The useful range spans orders of magnitude, which is why λ cannot be guessed.

Slide 43 — The unpenalised fit leaves the picture



The same three fits drawn on the data, which is the table's last column made visible. Look first at the vertical scale on the left: the unpenalised curve leaves the axes entirely between the training points and returns only where a point is there to pull it back. At = 0.01 the fit is indistinguishable from the true curve (dashed). At = 100 it is a nearly flat line — wrong, but wrong by a few hundred kWh rather than by thousands. That is the asymmetry the table hides: over-penalising is a mild failure, under-penalising is a catastrophic one.

Look at the vertical scale on the left: the unpenalised curve shoots off the top and the bottom between the training points, coming back only where a point pulls it. That is what a coefficient of three billion looks like from outside. In the middle, $\lambda = 0.01$: on top of the dashed true curve. On the right, $\lambda = 100$: a nearly flat line that misses the curve, but by a few hundred kilowatt-hours where the left panel misses by thousands. The asymmetry the table hides: too much penalty is a mild failure, too little a catastrophic one. If you must err, err towards more.

Slide 44 — Too flexible, too rigid



The trade, swept over ten orders of magnitude. Test error falls from 57 at = 10 to 14.6 at = 10 , then climbs to 228. Two things to read off it: the minimum is nowhere near the round number you would have guessed, and the floor is broad — anything between 10 and 10 ² is within a few kWh of the best. That flatness is why a penalty can be searched for rather than solved for, and Lesson 5 is how.

The trade over many values of λ. Both ends fail, for opposite reasons: on the left the model is too flexible (it follows the noise), on the right too rigid (it cannot follow the curve). This shape is the **bias-variance trade-off**, which lesson 5 treats in full. How to choose λ? Not by looking at this test curve, because choosing on the test set would make its score optimistic: by cross-validation, lesson 5.

Slide 45 — Ridge has an exact solution too

$$\theta = (X^\top X + \lambda I)^{-1} X^\top y$$

Put it next to the normal equation of slide 17: the only change is λ added down the diagonal of the matrix being inverted. Same derivation, same projection idea, same four lines of NumPy. Ridge is least squares with the floor of the bowl tilted.

Two details that the handout (section 6.2) works out. The λ in this formula is not the same number as the λ of slide 40: the cost averages the squared errors over the m examples but not the penalty, so the two are on different scales, and with the handout's half in front of the cost the factor between them is $2m$. A penalty value only means something beside the cost it belongs to. And written with θ, the formula would also penalise the intercept; in practice the intercept's entry of I is set to zero, or the data are centred first, which is what scikit-learn does. Lasso has no such formula: the absolute value has a corner at zero, and the solution must be found iteratively.

Worked example — one coefficient, four penalties

The three houses of slide 19 ($\sum xy = 26$, $\sum x^2 = 14$, no intercept). With one feature, $(X^\top X + \lambda I)^{-1} X^\top y$ is just a division:

λ	$w = \sum xy / (\sum x^2 + \lambda)$	w
0	$26/(14+0)$	**1.857**
1	$26/(14+1)$	**1.733**
14	$26/(14+14)$	**0.929**
100	$26/(14+100)$	**0.228**

$\lambda = 0$ is least squares. Adding λ to the denominator shrinks the coefficient; $\lambda = \sum x^2$ halves it; no finite λ makes it zero. "Tilting the floor" is literally this: the same bowl, pulled towards $w = 0$.

Slide 46 — Why that one change fixes it

Least squares breaks down when there is a direction in which moving the coefficients changes the predictions not at all, as with two identical columns: the valley is perfectly flat along it and there

is no unique lowest point. Ridge makes every direction cost something, so the flat floor acquires a slope and a single lowest point appears. The algebra in one sentence: adding λ to the diagonal shifts every eigenvalue of $X^\top X$ up by λ, and a matrix can be inverted exactly when none of its eigenvalues is zero. So for any $\lambda > 0$, **Ridge has a unique answer even where least squares has none**. Hoerl and Kennard introduced Ridge in 1970 for exactly this problem; curing overfitting was the side effect.

Worked example — the same column twice

Put the area of slide 19 in twice, $x_1 = x_2 = (1, 2, 3)$:

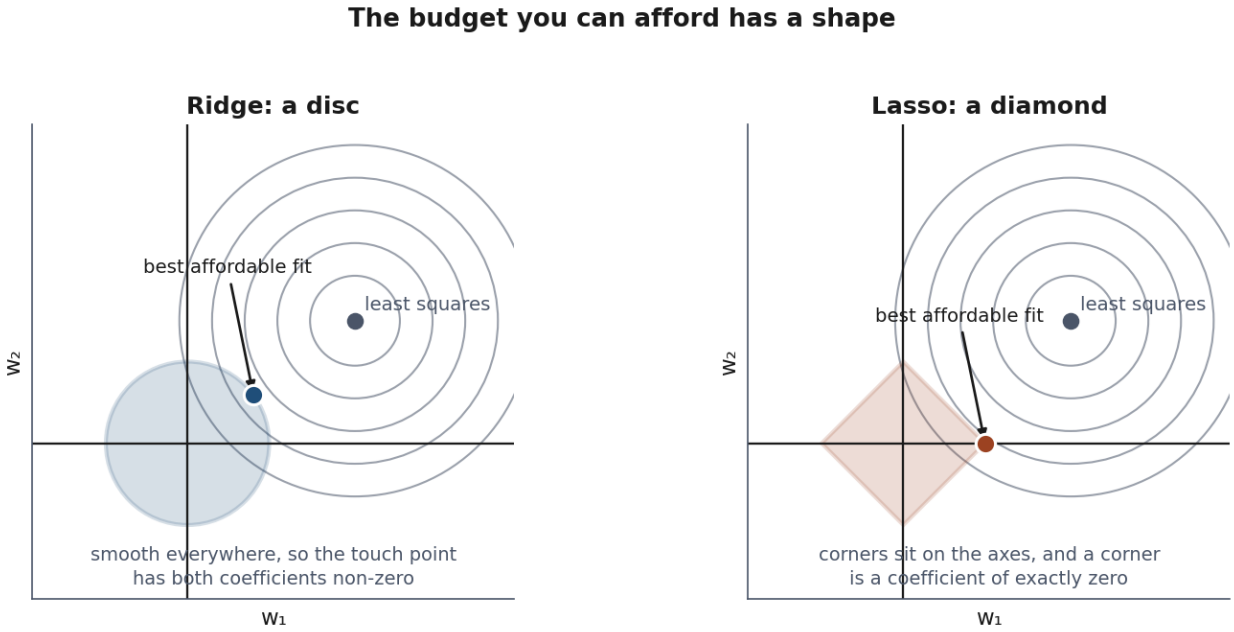
$$X^\top X = \begin{pmatrix} 14 & 14 \\ 14 & 14 \end{pmatrix}, \quad \det = 14 \cdot 14 - 14 \cdot 14 = 0$$

No inverse: eigenvalues 28 and 0, and the zero is the flat direction — raise w_1 and lower w_2 by the same amount and nothing changes. Add $\lambda = 1$ down the diagonal:

$$\begin{pmatrix} 15 & 14 \\ 14 & 15 \end{pmatrix}, \quad \det = 225 - 196 = 29$$

Both eigenvalues moved up by 1 — to 29 and 1 — so neither is zero and the inverse exists. Every $\lambda > 0$ does the same.

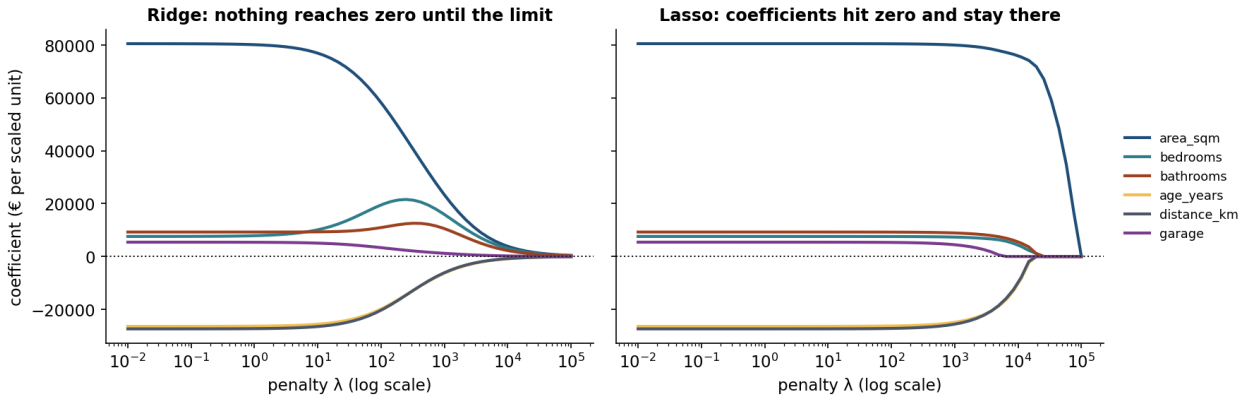
Slide 47 — The picture behind Ridge vs Lasso



*The constraint regions: a disc for Ridge, a diamond for Lasso. The diamond has corners **on the axes**, and a corner is where the solution tends to land — which is the whole reason Lasso produces exact zeros and Ridge does not.*

Think of the penalty as a **budget**: a fixed total you may spend on coefficients, and you want the best fit that money buys. Charging squares, the affordable region is a disc; charging absolute values, a diamond standing on its corners. The best affordable fit is where the error contours first touch the region. A disc is smooth, so the contact happens at an ordinary point of its edge, with both coefficients non-zero. A diamond has corners, and its **corners lie on the axes**, where one coefficient is exactly zero; corners stick out, so they tend to be touched first. That is the whole reason Lasso produces zeros and Ridge does not.

Slide 48 — Ridge shrinks, Lasso selects



Every coefficient tracked as the penalty sweeps from nothing to a lot. Ridge coefficients approach zero and arrive only in the limit; Lasso coefficients reach it at a finite penalty and stay there. What to look at on the left is `bedrooms`*, which* ***triples*** *before it falls: as the penalty pushes* `area_sqm` *down, the feature correlated with it at 0.77 picks up the slack. Shrinkage is not the same thing as each coefficient falling monotonically, and Section 7.2 is why.*

Every coefficient tracked as the penalty grows. Ridge coefficients approach zero smoothly and reach it only in the limit; Lasso coefficients hit zero at a finite λ and stay there. One detail worth noticing on the left: `bedrooms` *rises* before it falls, because as the penalty pushes `area_sqm` down, the feature correlated with it picks up part of its role. Shrinking does not mean that each coefficient falls steadily.

Worked example — three coefficients, both penalties

In the simplest case — features uncorrelated and of unit length, penalty written so the formulas come out clean — each penalty acts on each least-squares coefficient separately. Ridge **divides**, $w/(1+\lambda)$; Lasso **subtracts and stops at zero**, $\text{sign}(w)\max(|w|-\lambda, 0)$. Least squares gives 5, 2, 0.5:

λ	Ridge	Lasso
0	5.000, 2.000, 0.500	5.000, 2.000, 0.500
1	2.500, 1.000, 0.250	4.000, 1.000, 0.000
3	1.250, 0.500, 0.125	2.000, 0.000, 0.000

Ridge keeps all three, smaller. Lasso drops the smallest first and, by $\lambda = 3$, keeps one feature. The order it drops them in is the order of how much each was worth — which is slide 49's table.

Slide 49 — The order Lasso drops features

	Features kept
1,000	all six
10,000	five: `garage` dropped
20,000	three: `area_sqm`, `bedrooms`, `bathrooms`
40,000	one: `area_sqm`

The order is not arbitrary. Once every feature is on a common scale, `garage` buys the least accuracy per unit of coefficient spent, so it goes first; `area_sqm` buys the most and survives longest. Nobody told Lasso which features matter: it was given a budget and spent it where it bought the most. The warning that gets forgotten: **the selection depends entirely on** λ. "Lasso chose these features" is never a complete statement, and presenting Lasso's output as an objective ranking of importance is one of its commonest misuses.

Question — what is the "budget", and where is it in the cost?

> *Slide 49: when we speak of a "budget", what does it refer to?*
>
> *Clear, but the budget t is never made explicit. Minimising the regularised J we look for the coefficients that minimise both the MSE and the regularisation part, but it is a trade-off between the two: in general the MSE will be minimised less than without regularisation, in exchange for smaller coefficients. Correct?*

Correct on both counts. The **budget** is the total size of the coefficients the model is allowed to use: for Lasso, the sum of their absolute values. It comes from writing the same problem in a second form. Instead of adding a penalty,

$$\min_{w} \ \text{MSE}(w) + \lambda \sum_{j} |w_j|,$$

one can impose a limit:

$$\min_{w} \ \mathrm{MSE}(w) \quad \text{subject to} \quad \sum_{j} |w_j| \leq t.$$

The two forms give the same solutions (a result called Lagrange duality): for every penalty strength λ there is a budget t that produces the same coefficients, and the other way round. A larger λ corresponds to a smaller budget.

You are right that t never appears in what is actually computed: the slides and scikit-learn fix λ, not t, and the budget is set implicitly. The budget form is a way of *picturing* the problem, and it is what makes the disc and the diamond of slide 47 possible: you cannot draw a penalty, but you can draw the region of coefficients that fit within a budget.

And yes, it is a trade-off. Without the penalty the coefficients are chosen to make the training MSE as small as possible, so no other choice can have a lower training MSE; with the penalty, the minimum of the *sum* is generally somewhere else, with a higher training MSE and smaller coefficients. Slide 42 shows it: training RMSE 13.6 without a penalty, 18.0 at $\lambda = 0.01$, while the test RMSE falls from 24,655.7 to 22.8. The price is paid on the training data; the hoped-for gain is on new data.

Part VI — Reading coefficients

Slide 50 — The case Ridge was invented for

`area_sqm` and `area_sqft`: the same measurement in two units, correlation 0.999999, as when two systems are merged and the same fact arrives twice. Least squares has no basis for choosing between "all the effect on the metres column", "all of it on the feet column", and everything in between: they all fit identically. So it picks one arbitrarily, and the choice changes with the sample.

Worked example — one effect, two columns

The duplicated area of slide 46. Least squares only pins down the **sum** $w_1 + w_2 = 26/14 = 1.857$; every split predicts the same prices:

w_1	w_2	$w_1 + w_2$	$w_1^2 + w_2^2$
1.857	0.000	1.857	3.45
0.929	0.929	1.857	1.72
6.857	-5.000	1.857	72.02

Same fit, very different coefficients — and a solver will report whichever the arithmetic happens to land on. Ridge charges $w_1^2 + w_2^2$, which is smallest for the even split, so it chooses it: with $\lambda = 1$, $w_1 = w_2 = 26/29 = 0.897$, a sum of 1.793 — shrunk a little, and divided fairly.

Slide 51 — The coefficients are noise

Split	area_sqm	area_sqft
0	9,261	−640
1	14,535	−1,127
2	45,307	−3,989
3	39,061	−3,402

Same data, four random splits. The area coefficient varies by a factor of five, and its duplicate swings the opposite way to compensate. What is broken and what is not: the **predictions** are fine throughout - a square metre is 10.76 square feet, so the metres coefficient plus 10.76 times the feet coefficient is what a square metre adds, and on every split it stays within about 2% of the true 2,400 euros (2,424 on the notebook's own split) - but the **explanation** is noise. Anyone reading these coefficients to learn what drives house prices would draw nonsense, confidently. With Ridge at $\lambda = 10$ the same four splits give about 38,000 to 41,000 on both columns, split evenly, because two moderate coefficients cost less under a squared penalty than one huge positive and one huge negative. These are euros per standard deviation of area, because Ridge runs on scaled features, so they cannot be set against the euros per square metre in the table; what matters is that they are stable and equal. Ridge is not only a cure for overfitting: it makes coefficients readable when features overlap.

Slide 52 — What a coefficient actually says

A coefficient is the change in the prediction for a one-unit change in that feature, **with every other feature held fixed**. The last clause does enormous work. The coefficient on `bedrooms` is not "what a bedroom is worth"; it is what a bedroom is worth *to a house whose floor area does not change*, which means the existing space was subdivided. That is a different thing from an extra bedroom in an extra 15 square metres. This is also why two honest analysts can fit the same data, report different coefficients and both be right: they included different features, so their "held fixed" clauses differ.

Worked example — the same column, two coefficients

Price $= 2\,x_1 + 3\,x_2$ exactly, no noise, where $x_1 = 1, 2, 3, 4$ and $x_2 = 1, 3, 3, 5$ (correlated, $r = 0.95$).

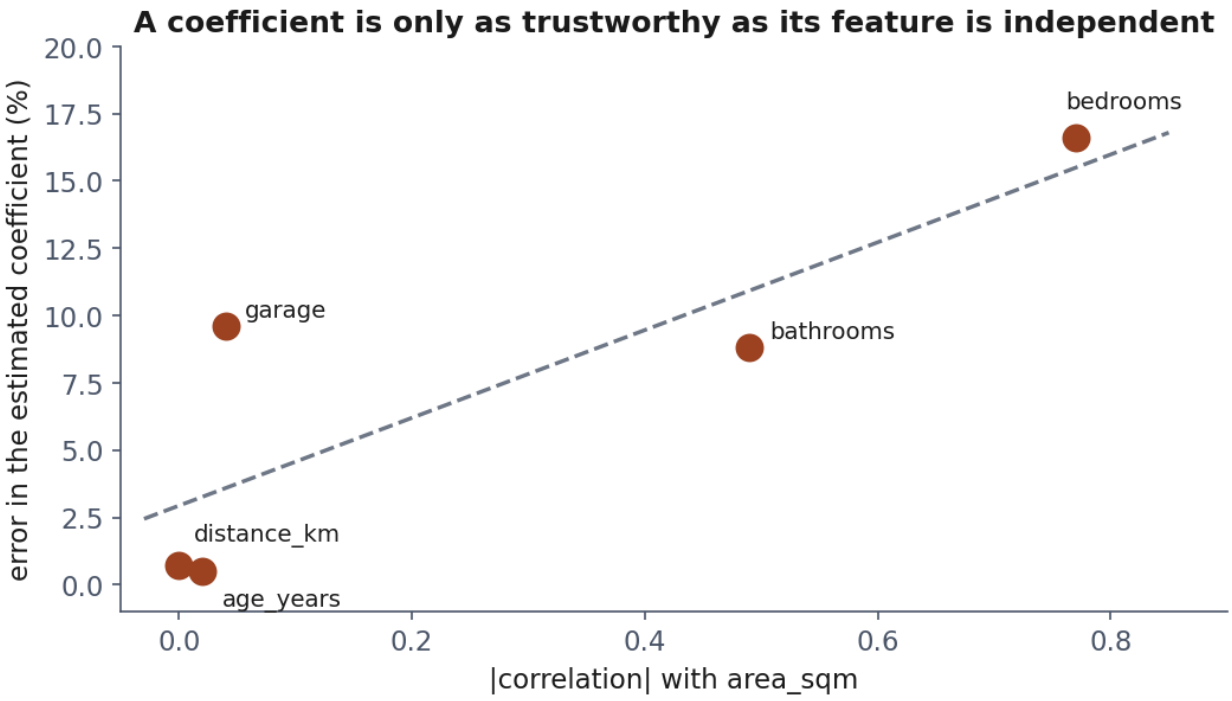
- Fit both: coefficients **2.0** and **3.0** — the truth.
- Leave x_2 out: the coefficient of x_1 becomes **5.6**.

Neither fit is wrong. With x_2 in the model, 2 answers "one more unit of x_1, x_2 held fixed". Without it, 5.6 answers "one more unit of x_1, *and whatever x_2 usually does alongside it*". The lesson's 2,785 €/m² (area alone) against 2,421 (all six features) is this, on the real data.

Slide 53 — When can you trust a coefficient?

From notebook 1, comparing each estimated coefficient with the truth:

Feature	True	Estimated	Error	Correlation with area
`area_sqm`	2,400	2,421	+0.9%	—
`age_years`	−1,800	−1,791	−0.5%	−0.02
`distance_km`	−6,500	−6,455	−0.7%	0.00
`garage`	12,000	10,850	−9.6%	−0.04
`bathrooms`	14,000	15,229	+8.8%	0.49
`bedrooms`	9,000	7,503	**−16.6%**	**0.77**



The table above, drawn. Each point is one feature: how far its estimated coefficient lands from the truth, against how strongly it correlates with `area_sqm`*. Two of the three features that share nothing with area are recovered to within 1%; the two that share a great deal are out by 9% and 17%.* `garage`

is the honest exception — uncorrelated and still 9.6% out — and it is there to stop the line being read as a law.

The two features correlated with area are out by 9% and 17%, and the worst estimate belongs to the most correlated one; age and distance, uncorrelated, are within 1%. `garage` is the honest exception, uncorrelated and still 9.6% out: a yes-or-no column varies little from house to house, so the noise weighs more on its coefficient.

Question — what does the note of slide 53 say?

> *Explain the note of slide 53 better: "the x axis is how correlated each feature is with area; the y axis is how far its estimated coefficient landed from the truth… state the rule: a coefficient is only as trustworthy as its feature is independent… a coefficient is an association, not a causal effect."*

The note makes two separate points.

First: correlated features make coefficients hard to pin down. Suppose bigger houses almost always have more bedrooms. A house that has both 40 more square metres *and* one more bedroom costs more, but how much of the difference is due to the area and how much to the bedroom? With the true values, 2,400 × 40 + 9,000 = 105,000 euros. But 2,500 × 40 + 5,000 also gives 105,000, and so does 2,000 × 40 + 25,000. If the two features always moved together, every one of these splits would fit the data equally well, and the data could not tell them apart. They do not move together perfectly here (correlation 0.77, not 1), so the fit can separate them, but only imprecisely: what separates them is the houses where area and bedrooms vary independently, and the more correlated the features, the fewer such houses there are. That is the meaning of "a coefficient is only as trustworthy as its feature is independent", and it is slide 21's condition number again: nearly dependent columns make $X^\top X$ hard to invert, and its inverse governs how uncertain the coefficients are.

On a real dataset the true coefficient is unknown, so the same wobble would be there with nothing to reveal it: you would see "7,503 per bedroom" and might believe it. That is why the stability of a coefficient has to be checked, for instance by refitting on different samples as slide 51 did.

Second: even a well-estimated coefficient is an association, not a cause. If houses near the centre are both older and more expensive, age and distance share out the price difference in whatever proportion fits the sample best. Neither number says what would happen if you moved a house, or aged it: other things travel with location (services, transport, the neighbourhood) that the model never saw. A coefficient answers "how do prices differ between houses that differ in this feature, all else in the model equal", not "what would happen if I changed this feature".

Question — ideally, should features be uncorrelated with each other and correlated with the label?

> *Still on slide 53: could we say that in theory the ideal would be to have features all uncorrelated with each other, but with some correlation with the label?*

As an intuition for **readable coefficients**, yes: features that each bring their own information, not already carried by the others, give the fit a clear basis for attributing the effect, and the coefficients come out stable. In the ideal case of standardised, uncorrelated features, $X^\top X$ is a multiple of the identity, the condition number is 1, and each coefficient can be estimated independently of the others.

Two refinements make the statement precise. First, a feature need not be *linearly correlated* with the label to be useful: if $y = x^2$ with x symmetric around zero, the correlation between x and y is zero, yet x determines y completely (the squared column of slide 32 is how a linear model uses it). A better phrase is "informative about the label". Second, correlated features are a problem for *interpreting* coefficients, not necessarily for *predicting*: slides 50-51 showed predictions staying accurate while the two area coefficients swung wildly. So the ideal is better put as: **features that carry useful information about the label, without repeating one another.** When they do repeat one another, Ridge is the tool.

Slide 54 — Notebook 3, live

The table maps the notebook to the slides and calls. The collinear section is the one to reach: it is the argument to have ready whenever someone asks what a model says drives the outcome. The "try this" swaps Ridge for Lasso on the two area columns: predict which way Lasso resolves the redundancy before running it, then explain why the two penalties disagree. It checks whether the disc and the diamond of slide 47 have landed.

Slide 55 — What we did today

The thread: lesson 2 prepared data honestly; today we fitted something to it and found that fitting well and explaining well are different goals. The model that fits the training data best is usually not the one you want. And the number to remember: a penalty of one hundredth took the largest coefficient from about three billion to **365**, and the test error from 24,656 to 22.8. Lesson 4 keeps the same machinery but makes the target a category instead of a number, which turns out to need a different cost function.

Slide 56 — Homework: we discuss it on Friday 23 October

Exercise 3 uses a new dataset, bike-sharing demand, so the exploration is yours to do. Two tasks carry the most weight. Task 4 asks for a choice of λ justified without looking at the test set: awkward before lesson 5, on purpose, so that the need for cross-validation is felt before it is taught. Task 6 asks which coefficients you trust and why, which is Part VI applied. At the exam, as always, the reasoning counts, not the accuracy.